

University of Bucharest
Faculty of Mathematics and Computer Science
Artificial Intelligence - Machine Learning

Project Report

Radio Signal Image Classification

Chiritoiu Andra-Florentina

Group 234

1 Description of the Problem and Data	2
2 General Methodology	2
3 Model 1: K-Nearest Neighbors baseline	4
3.1 Idea and Preprocessing	4
3.2 Model and Hyperparameters	4
4 Model 2: Linear SVM on Raw Pixels	5
4.1 Idea and Preprocessing	5
4.2 Hyperparameters and Results	5
5 Model 3: Extra Trees Classifier	6
5.1 Idea	6
5.2 Results	6
6 Model 4: HOG + Linear SVM	7
6.1 Idea	7
6.2 Results	7
7 Model 5: HOG + Global Features + Hough + Linear SVM	8
7.1 Idea and Features	8
7.2 Results	8
8 Model 6: HOG + Global Features + Hough + Softmax	9
8.1 Idea	9
8.2 Results	9
9 Model 7: XGBoost with Extended Feature Extraction	10
9.1 Idea	10
9.2 Results	10
10 Model 8: XGBoost with Feature Extraction + PCA	11
10.1 Motivation	11
10.2 Results	11
11 Model 9: Custom CNN RGB + EDGE	12
11.1 Motivation	12
11.2 Preprocessing	12
11.3 Architecture and Training	12
11.4 Results	12
12 Model 10: DeepSignalCNN with SE + ASPP - RGB + EDGE + FREQ	14
12.1 Motivation	14
12.2 Preprocessing and Training	14
12.3 Results and Ensemble	14
13 Advanced CNN Models and Final Ensemble	15
13.1 GREEN + Sliding Window 1x7	15
13.2 2D-CNN with Asymmetric Pooling and 1D-Profile-CNN	15
13.3 Final Ensemble	15
14 Comparison of All Methods	16
15 Conclusions	17

Summary

This report documents the approach used for the radio signal image classification competition. The task is a multi-class classification problem with 5 classes. Classical machine learning methods, manual feature extraction methods, and convolutional models trained from scratch were tested. The report presents the data used, preprocessing, augmentations, feature representations, tested hyperparameters, validation-set results, and confusion matrices. The best solution obtained was an ensemble of two 2D-CNN models and one 1D-Profile-CNN model, with a validation accuracy of 0.7571 (all values reported in this document represent locally computed accuracies; Kaggle scores were generally higher, by up to approximately 0.03).

The report includes all approaches that were attempted, including unsuccessful or weaker variants.

1 Description of the Problem and Data

The competition task is to classify radio signal images into one of 5 classes. Each example is a .png image, and the train.csv and test.csv files contain the image names and, for the training set, the associated labels. The dataset used has the following structure:

Table 1: Dataset Structure

Set	Number of images
Train	15500
Test	5500
Number of classes	5

The class distribution in the training set is:

Table 2: Class Distribution in the Training Set

Original class	Number of images
1	3500
2	3000
3	3000
4	3000
5	3000

For local evaluation, the data were stratified into 80% train and 20% validation:

Table 3: Local Train-Validation Split

Subset	Number of images
Local train	12400
Validation	3100

In the code, labels were internally used as values 0, 1, 2, 3, 4, corresponding to the original classes 1, 2, 3, 4, 5. For the CNN models, because the first class has more examples, class weights were used in the loss function:

Table 4: Class Weights Used for CNN Models

Class	Weight
1	0.8857
2	1.0333
3	1.0333
4	1.0333
5	1.0333

2 General Methodology

I started with simple models to verify the image loading, preprocessing, training, validation, and submission-file generation pipeline. I then increased the complexity through manual feature extraction, classical ensemble models, and convolutional neural networks.

Convolutional neural networks.

The general workflow was:

1. reading images from .png files;
2. splitting the data into local train and validation sets;
3. applying model-specific preprocessing;
4. training the models;
5. selecting hyperparameters on the validation set;
6. analyzing confusion matrices;
7. generating predictions for the test set.

The following model families were tested:

- models on raw pixels: KNN, Linear SVM, Extra Trees;
- models with manual feature extraction: HOG, LBP, histograms, projections, FFT, connected components, Hough;
- linear models on manually extracted features: Linear SVM and Softmax;
- XGBoost on extended features;
- XGBoost with PCA;
- custom CNNs trained from scratch;
- ensembles of CNN models.

All final models comply with the restriction of not using external data, pretrained models, or transfer learning.

3 Model 1: K-Nearest Neighbors baseline

3.1 Idea and Preprocessing

The first approach was K-Nearest Neighbors, used as a baseline to verify the entire pipeline. The images were:

- read using the PIL library;
- converted to grayscale;
- resized to 64×64 ;
- normalized by dividing pixel values by 255.0;
- transformed into 1D vectors using `flatten()`.

For 64×64 , each image becomes a vector with $64 \cdot 64 = 4096$ features.

3.2 Model and Hyperparameters

KNN classifies a new image by comparing it with images from the training set. For each validation image, the model computes the distance to all training images, selects the k nearest neighbors, and returns the majority class.

The L1 and L2 distances were tested:

$$d_{L1}(x, y) = \sum_i |x_i - y_i|$$

$$d_{L2}(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

Table 5: KNN Results on Validation

Model	Image size	Metric	k	Validation accuracy
KNN	64×64	L2	1	0.2100
KNN	64×64	L2	3	0.2155
KNN	64×64	L2	5	0.2200
KNN	64×64	L2	7	0.2158
KNN	64×64	L2	9	0.2216
KNN	64×64	L1	1	0.2010
KNN	64×64	L1	3	0.2148
KNN	64×64	L1	5	0.2223
KNN	64×64	L1	7	0.2132
KNN	64×64	L1	9	0.2158

The best configuration was $k = 5$, L1 metric, with an accuracy of 0.2223. The result is weak because the model compares pixels directly and does not learn abstract visual structures.

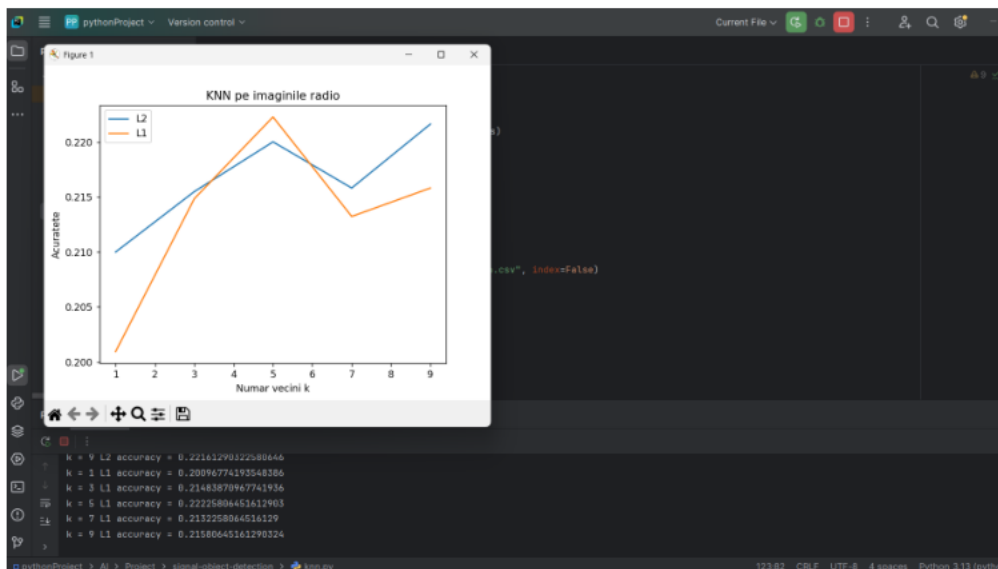


Figure 1: KNN tuning results: validation accuracy for L1 and L2 distances as a function of the number of neighbors.

4 Model 2: Linear SVM on Raw Pixels

4.1 Idea and Preprocessing

The second model was a Linear SVM applied to raw pixel vectors. Image preprocessing was the same as for KNN: grayscale, resize to 64×64, normalization to [0, 1], and vectorization. In addition, StandardScaler was used, fitted only on the local training set and then applied to validation.

4.2 Hyperparameters and Results

The main parameter tested was C , which controls the trade-off between maximizing the margin and penalizing errors.

Table 6: Linear SVM Results on Raw Pixels

Model	Image size	Features	C	Validation accuracy
Linear SVM	64×64	4096	0.001	0.2048
Linear SVM	64×64	4096	0.01	0.1984
Linear SVM	64×64	4096	0.1	0.1997
Linear SVM	64×64	4096	1	0.1984
Linear SVM	64×64	4096	10	0.1977

The best configuration was $C = 0.001$, with an accuracy of 0.2048. The model did not outperform KNN, showing that raw pixels are not a sufficient representation for this problem.

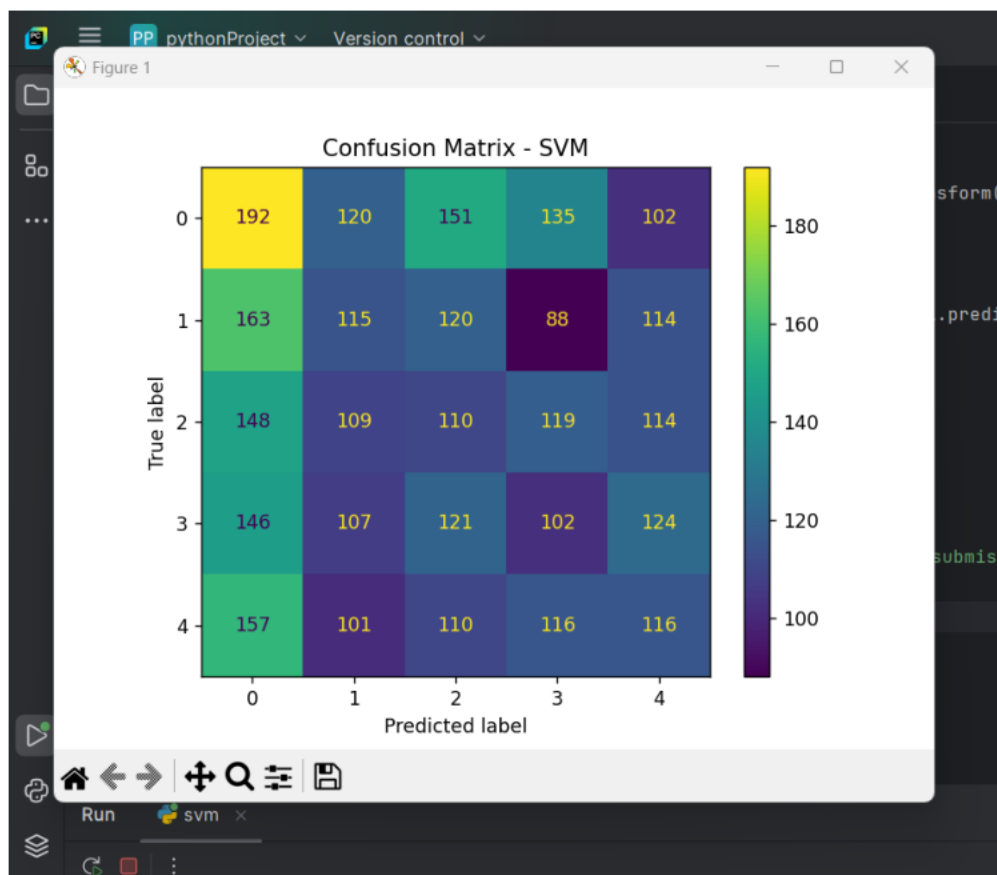


Figure 2: Confusion matrix for Linear SVM on raw pixels, computed on the validation set.

5 Model 3: Extra Trees Classifier

5.1 Idea

Extra Trees is an ensemble model based on multiple decision trees. Each tree learns different rules, and the final prediction is obtained by majority vote. The model introduces more randomization than a standard tree by randomly selecting certain features and split thresholds.

5.2 Results

Table 7: Extra Trees Results

Model	n_estimators	max_depth	Validation accuracy
Extra Trees	100	None	0.2600
Extra Trees	100	20	0.2535
Extra Trees	100	40	0.2684
Extra Trees	300	None	0.2665
Extra Trees	300	20	0.2587
Extra Trees	300	40	0.2558
Extra Trees	500	None	0.2616
Extra Trees	500	20	0.2600
Extra Trees	500	40	0.2645

The best configuration was n_estimators=100, max_depth=40, with a validation accuracy of 0.2684. Extra Trees was better than KNN and Linear SVM on raw pixels, but performance remained low.

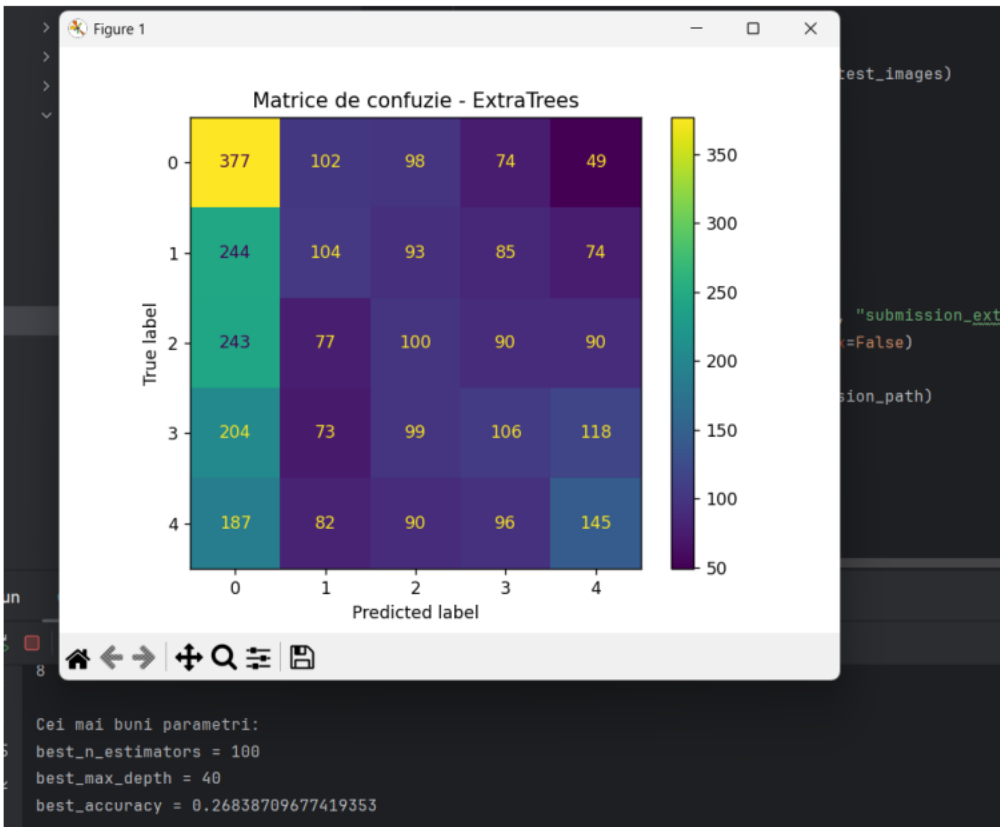


Figure 3: Confusion matrix for Extra Trees Classifier, computed on the validation set.

6 Model 4: HOG + Linear SVM

6.1 Idea

The previous models used the image pixels directly. To extract more relevant features, the HOG descriptor, Histogram of Oriented Gradients, was used. HOG extracts information about edges, contours, and local orientations.

The pipeline was:

Image $\rightarrow$ grayscale $\rightarrow 64 \times 64 \rightarrow$ HOG $\rightarrow$ StandardScaler $\rightarrow$ LinearSVC.

The HOG parameters used were:

Table 8: HOG Parameters

Parameter	Value
orientations	12
pixels_per_cell	(8, 8)
cells_per_block	(2, 2)
block_norm	L2-Hys
Image size	64×64

6.2 Results

Table 9: HOG + Linear SVM Results

Model	C	Validation accuracy
HOG + Linear SVM	0.00005	0.2981
HOG + Linear SVM	0.0001	0.2997
HOG + Linear SVM	0.0002	0.3026
HOG + Linear SVM	0.0003	0.3026
HOG + Linear SVM	0.0004	0.3003

$C = 0.0002$ was selected because it achieves the same score as $C = 0.0003$, but with slightly stronger regularization.

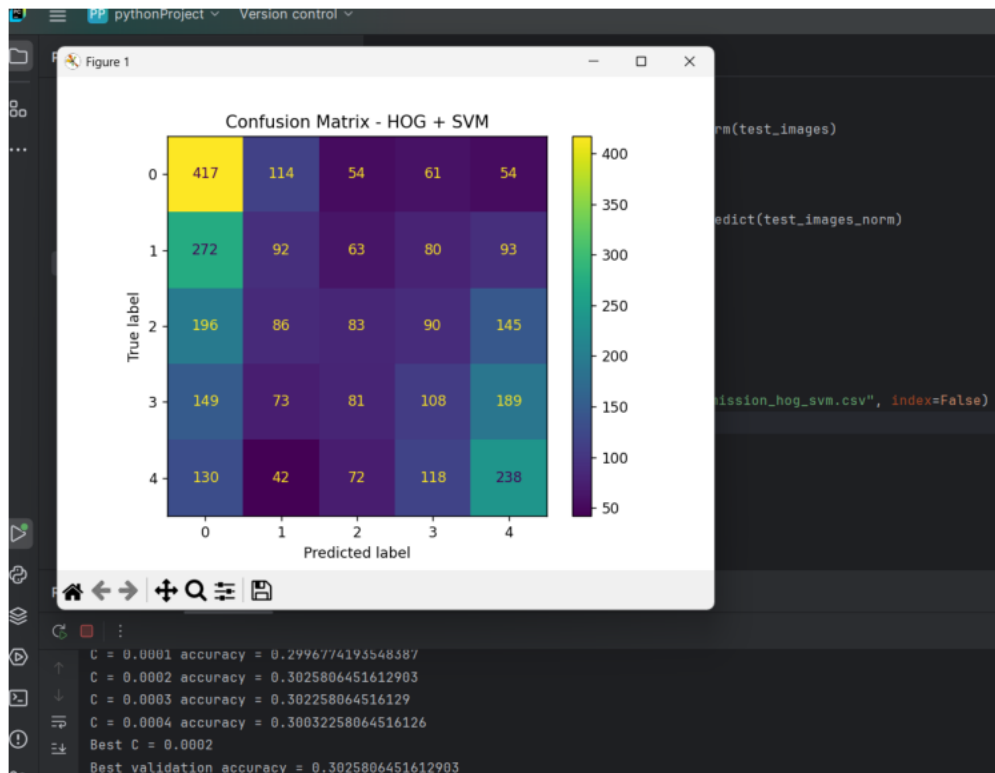


Figure 4: Confusion matrix for HOG + Linear SVM, computed on the validation set.

7 Model 5: HOG + Global Features + Hough + Linear SVM

7.1 Idea and Features

After simple HOG, additional features suitable for visual radio signals were added:

- global pixel-intensity statistics;
- distribution of bright pixels across rows and columns;
- number and area of connected components;
- Hough features for line detection.

In this configuration, each image was represented by 2449 features.

7.2 Results

Table 10: HOG + Global + Hough + Linear SVM Results

Model	class_weight	C	Validation accuracy
Linear SVM	None	$1 \cdot 10^{-6}$	0.2965
Linear SVM	None	$3 \cdot 10^{-6}$	0.3132
Linear SVM	None	$5 \cdot 10^{-6}$	0.3190
Linear SVM	None	$6 \cdot 10^{-6}$	0.3213
Linear SVM	None	$7 \cdot 10^{-6}$	0.3252
Linear SVM	None	$8 \cdot 10^{-6}$	0.3245
Linear SVM	None	$9 \cdot 10^{-6}$	0.3252
Linear SVM	None	$1.2 \cdot 10^{-5}$	0.3252
Linear SVM	balanced	$5 \cdot 10^{-6}$	0.3203
Linear SVM	balanced	$6 \cdot 10^{-6}$	0.3226
Linear SVM	balanced	$7 \cdot 10^{-6}$	0.3252
Linear SVM	balanced	$1 \cdot 10^{-5}$	0.3216

The best configuration was $C = 7 \cdot 10^{-6}$, without class_weight, with an accuracy of 0.3252.

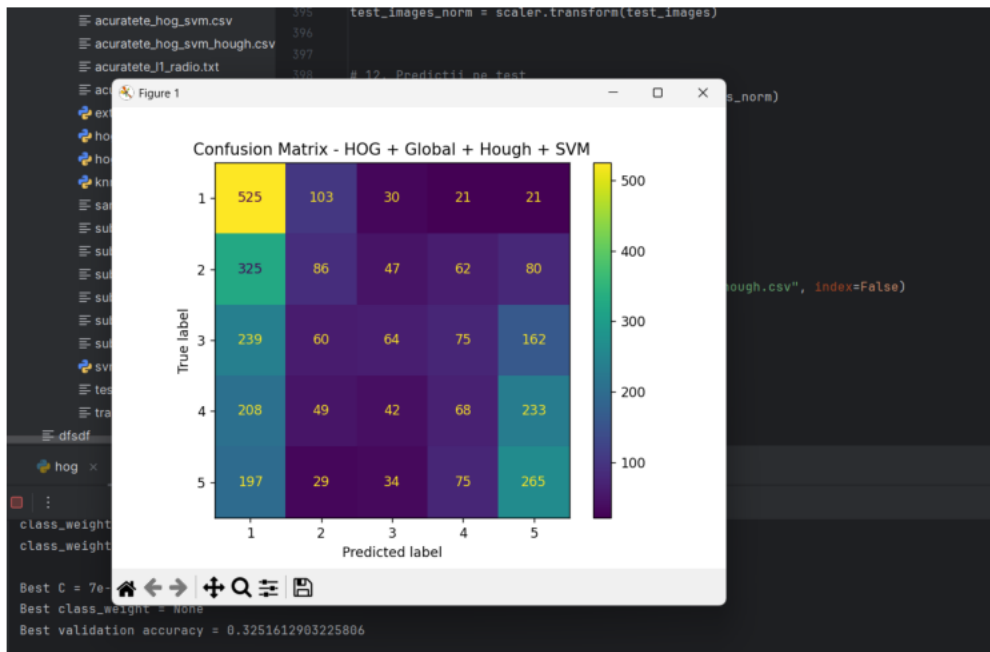


Figure 5: Confusion matrix for HOG + Global Features + Hough + Linear SVM, computed on the validation set.

8 Model 6: HOG + Global Features + Hough + Softmax

8.1 Idea

A Softmax classifier implemented through LogisticRegression was also tested. This is a linear multi-class model, similar to Linear SVM, but it optimizes a different loss function.

8.2 Results

Table 11: Softmax Results on Manual Features

Model	C	Validation accuracy
Softmax	0.0001	0.3342
Softmax	0.0003	0.3229
Softmax	0.0007	0.3197
Softmax	0.001	0.3206

Softmax achieved 0.3342 and outperformed the HOG + Global + Hough + Linear SVM variant.

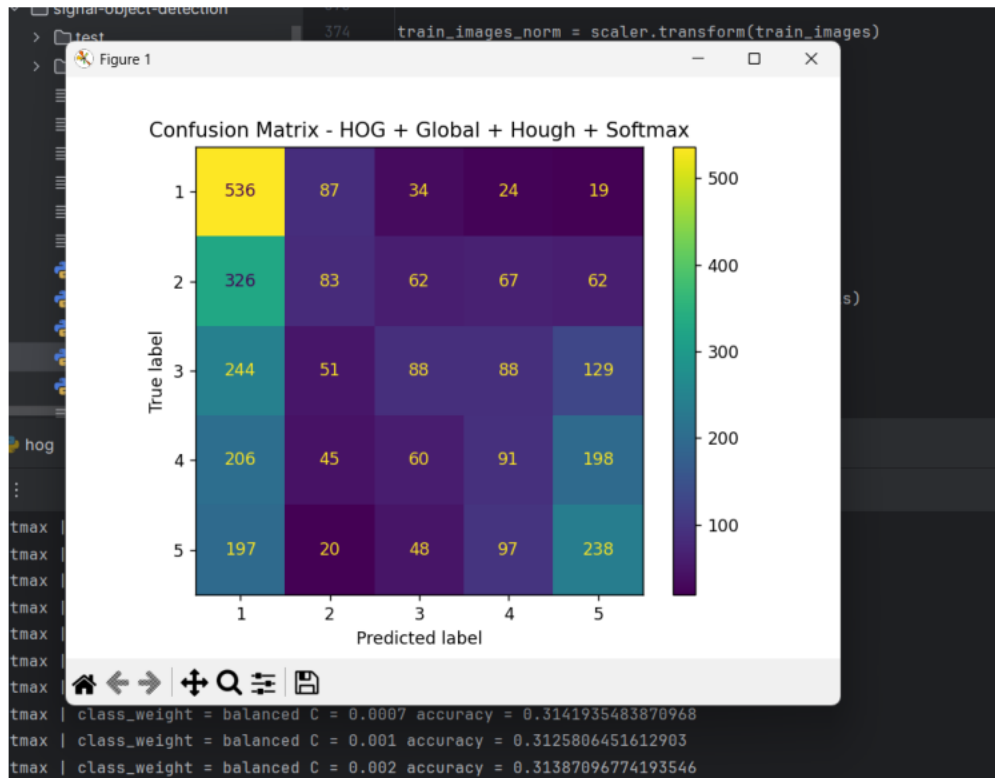


Figure 6: Confusion matrix for HOG + Global Features + Hough + Softmax, computed on the validation set.

9 Model 7: XGBoost with Extended Feature Extraction

9.1 Idea

XGBoost was tested because it can learn nonlinear relationships between features. For this model, the most extensive manual representation of the images was used:

- HOG;
- LBP;
- histogram of intensity;
- projections across rows and columns;
- FFT;
- connected components;
- Hough features.

StandardScaler was not applied to XGBoost because tree-based models are not sensitive to feature scale in the same way as linear models.

9.2 Results

Table 12: XGBoost Results with Extended Feature Extraction

n_estimators	max_depth	learning_rate	subsample	colsample	reg_lambda	Accuracy
300	2	0.03	0.9	0.9	3	0.3758
300	3	0.03	0.9	0.9	3	0.3968
500	3	0.05	0.9	0.8	3	0.4094
500	4	0.03	0.8	0.8	5	0.4206

XGBoost without PCA was the best classical method, with a validation accuracy of 0.4206.

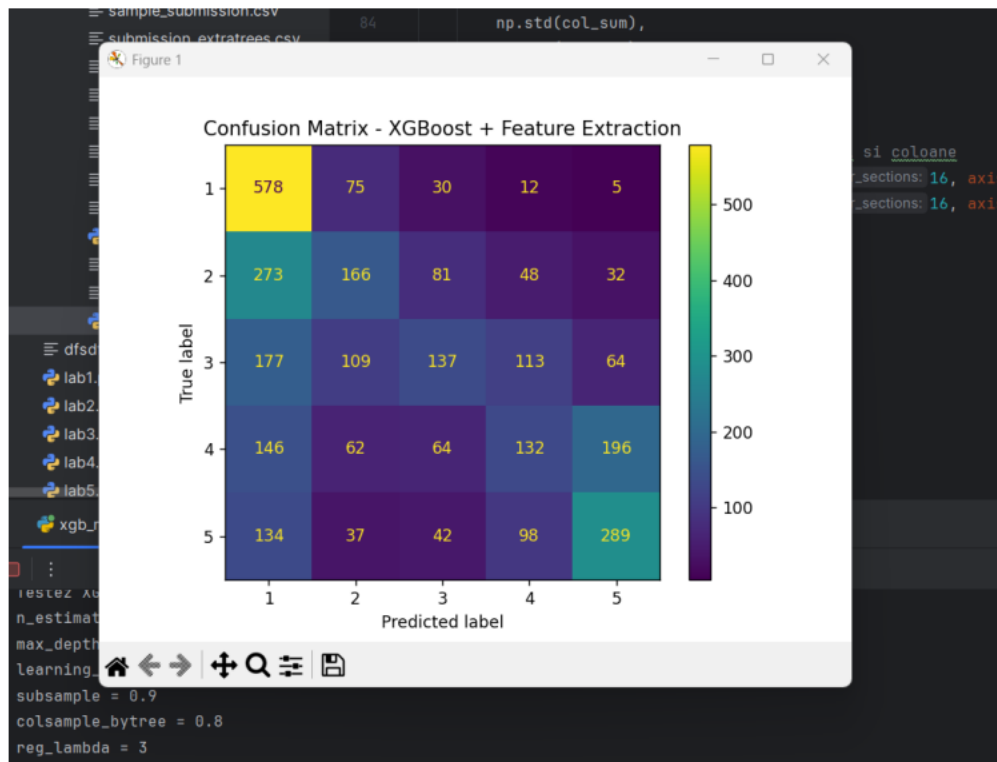


Figure 7: Confusion matrix for XGBoost with extended feature extraction, computed on the validation set.

10 Model 8: XGBoost with Feature Extraction + PCA

10.1 Motivation

After XGBoost without PCA, dimensionality reduction with PCA was tested. The goal was to reduce the number of features, remove noise, and obtain a more compact representation. Before PCA, StandardScaler was used, fitted only on the local training set.

The pipeline was:

Imagine → feature extraction extins → StandardScaler → PCA → XGBoost.

10.2 Results

Table 13: XGBoost + PCA Results

PCA	Number of components	Explained variance	Best validation accuracy
50	50	0.5819	0.3513
100	100	0.6547	0.3458
200	200	0.7363	0.3577
0.95	798	0.9500	0.3355

The best variant was PCA with 200 components, with an accuracy of 0.3577. However, this was worse than XGBoost without PCA, suggesting that PCA also removed useful information for separating the classes.

11 Model 9: Custom CNN RGB + EDGE

11.1 Motivation

After the classical models, a custom CNN architecture was implemented and trained entirely from scratch. The goal was to use a model that automatically learns relevant visual features directly from images. No pretrained models or external data were used. Two variants were trained:

- RGB model, on the original images;
- EDGE model, using the grayscale + Sobel + Laplacian representation.

11.2 Preprocessing

Images were resized to 128×128 . For the RGB model, the image was kept in RGB format. For the EDGE model, the three channels were:

Table 14: Channels for the EDGE Representation

Channel	Description
1	image grayscale
2	magnitudea Sobel
3	valoarea absoluta a filtrului Laplacian

Augmentations such as RandomHorizontalFlip, RandomAffine, ColorJitter, RandomErasing, and Normalize were used.

11.3 Architecture and Training

The ScratchSignalCNN model contains Conv-BN-SiLU blocks, residual blocks, Dropout2d, Adaptive Average Pooling, and a fully connected classifier. The main training settings were:

Table 15: ScratchSignalCNN Training Settings

Component	Value
Loss	CrossEntropyLoss with label smoothing
Optimizer	AdamW
Learning rate	$3 \cdot 10^{-4}$
Weight decay	$1 \cdot 10^{-4}$
Scheduler	CosineAnnealingLR
Batch size	64
Max epochs	70
Patience	14
Image size	128×128

11.4 Results

Table 16: Custom CNN RGB + EDGE Results

Model	Best epoch	Validation accuracy
ScratchSignalCNN RGB	60	0.6881
ScratchSignalCNN EDGE	52	0.6713
RGB + EDGE ensemble	-	0.6881

The ensemble did not improve the RGB model because the best weight found was 1.00 for RGB and 0.00 for EDGE.

Table 17: Confusion Matrix - ScratchSignalCNN RGB

True class	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	664	19	3	11	3
1	145	387	43	15	10
2	80	87	363	57	13
3	58	37	86	342	77
4	54	15	36	118	377

12 Model 10: DeepSignalCNN with SE + ASPP - RGB + EDGE + FREQ

12.1 Motivation

A more complex architecture, called DeepSignalCNN, was tested and trained from scratch. It combines three representations:

- RGB: original image;
- EDGE: grayscale + Sobel + Laplacian;
- FREQ: frequency-domain representation.

The model uses convolutional blocks, Batch Normalization, SiLU activations, residual connections, Squeeze-and-Excitation, and ASPP. Each model has 11,751,333 trainable parameters.

12.2 Preprocessing and Training

For FREQ, the image was converted to grayscale and a 2D Fourier transform was applied. The three FREQ channels were the logarithmic magnitude for low frequencies, the logarithmic magnitude for high frequencies, and the cosine of the phase. The main settings were:

Table 18: DeepSignalCNN RGB + EDGE + FREQ Settings

Component	Value
Image size	128×128
Batch size	48
Max epochs	80
Patience	18
Learning rate	$2 \cdot 10^{-4}$
Weight decay	$2 \cdot 10^{-4}$
Label smoothing	0.08
GPU	NVIDIA A100-SXM4-80GB

12.3 Results and Ensemble

Table 19: DeepSignalCNN Results on Multiple Representations

Model	Best epoch	Validation accuracy
RGB	77	0.6987
EDGE	57	0.6800
FREQ	26	0.2829
RGB + EDGE + FREQ ensemble	-	0.6997

The temperatures used for temperature scaling were 1.0504 for RGB, 0.8763 for EDGE, and 1.4859 for FREQ. The best weight combination was:

$$0.85 \cdot p_{RGB} + 0.05 \cdot p_{EDGE} + 0.10 \cdot p_{FREQ}.$$

Table 20: Confusion Matrix - RGB + EDGE + FREQ Ensemble

True class	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	685	13	1	1	0
1	160	400	20	13	7
2	98	73	359	47	23
3	80	27	76	349	68
4	60	18	26	120	376

13 Advanced CNN Models and Final Ensemble

This section groups the final variants that were tested, to avoid repetition and keep the report compact. All models in this section were trained from scratch, without external data, pretrained models, or transfer learning. The goal was to adapt the architecture to the structure of radio signal images, where lines, bright bands, and different distributions along the vertical and horizontal axes appear.

13.1 GREEN + Sliding Window 1x7

The first improvement over the RGB + EDGE + FREQ ensemble was using the green channel and a local 1×7 window. The three input channels were: the original green channel, the local mean over the 1×7 window, and the difference between the green channel and the local mean. The model was trained for 90 epochs, with an initial learning rate of $2 \cdot 10^{-4}$, batch size 48, and patience 20. The best result was a validation accuracy of 0.7139.

Table 21: GREEN + Sliding Window 1x7 Results

Epoch	Validation accuracy
10	0.4687
20	0.6506
30	0.6816
49	0.7052
75	0.7106
90	0.7139

Table 22: Confusion Matrix - GREEN + Sliding Window 1x7

True class	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	670	25	3	2	0
1	143	409	33	11	4
2	88	74	389	38	11
3	66	28	89	359	58
4	54	18	30	112	386

13.2 2D-CNN with Asymmetric Pooling and 1D-Profile-CNN

The second major improvement was a 2D-CNN with asymmetric pooling. It preserves directional signal structures better than symmetric pooling. In parallel, a 1D-Profile-CNN was tested, using profiles extracted from the image, such as the sum or mean of pixels across rows and columns. The 1D model was weak individually, but contributed a small amount to the ensemble.

Table 23: 2D-CNN and 1D-Profile-CNN Results

Model	Parameters	Validation accuracy
2D-CNN asymmetric pool	12.044.069	0.7558
1D-Profile-CNN	5.707.077	0.3255
2D-CNN + 1D-Profile-CNN ensemble	-	0.7561

The best intermediate combination was 90% 2D-CNN and 10% 1D-Profile-CNN. The confusion matrix showed that the model recognizes class 0 very well, but still confuses the similar classes 3 and 4.

13.3 Final Ensemble

For the final submission, the main 2D-CNN model was kept and a second 2D-CNN trained with a different seed, SEED_EXTRA=2026, was added. The third model in the ensemble was the 1D-Profile-CNN, retained for complementary information. Before combining the predictions, temperature scaling was applied to calibrate the probabilities.

Table 24: Models Used in the Final Ensemble

Model	Individual Val. acc.	Temperature	Final weight
2D-CNN old	0.7542	1.1755	0.65
2D-CNN extra seed2026	0.7435	1.2004	0.10
1D-Profile-CNN	0.2323	1.8802	0.25

The final prediction was calculated as a weighted average:

$$p_{final} = 0.65 \cdot p_{2D-old} + 0.10 \cdot p_{2D-extra} + 0.25 \cdot p_{1D-profile}.$$

The final validation result was 0.7571, the best local score obtained in the project.

Table 25: Confusion Matrix - Final Ensemble

True class	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	661	23	10	5	1
1	118	455	15	8	4
2	54	76	446	16	8
3	35	28	107	393	37
4	21	24	50	113	392

Table 26: Classification Report - Final Ensemble

Class	Precision	Recall	F1-score	Support
0	0.7435	0.9443	0.8320	700
1	0.7508	0.7583	0.7546	600
2	0.7102	0.7433	0.7264	600
3	0.7346	0.6550	0.6925	600
4	0.8869	0.6533	0.7524	600
Accuracy	0.7571			
Macro avg	0.7652	0.7509	0.7516	3100
Weighted avg	0.7645	0.7571	0.7542	3100

The most important confusions were: class 3 predicted as 2 in 107 cases, class 4 predicted as 3 in 113 cases, and class 2 predicted as 1 in 76 cases. These errors are explained by the visual similarity between consecutive classes. For the test set, 8 TTA predictions were used for each component, and the final generated file was submission_LAST_BEST.csv.

14 Comparison of All Methods

Table 27: Summary of Tested Methods

Method	Validation accuracy
KNN	0.2223
Linear SVM on raw pixels	0.2048
Extra Trees on raw pixels	0.2684
HOG + Linear SVM	0.3026
HOG + Global + Hough + Linear SVM	0.3252
HOG + Global + Hough + Softmax	0.3342
XGBoost with manual feature extraction	0.4206
XGBoost + PCA	0.3577
CNN custom RGB	0.6881
DeepSignalCNN RGB	0.6987
DeepSignalCNN RGB + EDGE + FREQ ensemble	0.6997
DeepSignalCNN GREEN + Sliding Window 1×7	0.7139
2D-CNN asymmetric pool	0.7558
2D-CNN + 1D-Profile-CNN ensemble	0.7561
2D old + 2D extra + 1D profile ensemble	0.7571

Compared with classical methods, CNN models brought a significant performance increase. The largest improvement occurred when the model was adapted to the structure of the images through asymmetric pooling and the use of an ensemble with differently trained models.

15 Conclusions

Several approaches were tested in the project. Simple models, such as KNN and Linear SVM on raw pixels, were useful for verifying the pipeline but had poor performance. Manual feature extraction using HOG, LBP, histograms, projections, FFT, connected components, and Hough improved the results, while XGBoost was the best classical method. The largest performance increase came from using convolutional neural networks trained from scratch. These models automatically learned relevant filters and visual representations for radio signal images. The best solution was the final ensemble of 2D-CNN old, 2D-CNN extra seed2026, and 1D-Profile-CNN. The final validation result was:
